

Title:- RangeGraph: Augmented Dynamic Graph Maintenance via Lazy Segment Trees and Binary Indexed Trees

1. **Pre-requisites:**

- a. Arrays & Indices
- b. Big-O notations
- c. Graph Basics
- d. Prefix Sums
- e. Binary Trees
- f. Binary Indexed Trees
- g. Segment Trees
- h. Lazy Propagations
- i. Graph adjacency representations
- j. Spanning Trees
- k. Connectivity Queries
- l. Lexicographic Ordering

2. **Motivation:** Modern networked systems like road networks, social graphs, communication networks, and biological pathways have one thing in common: they are always changing. Edges are temporary. Weights change. Parts separate and join. One of the main problems in modern algorithmics is how to answer queries about such changing structures meaningfully in real time.

For example, the Dhaka City Road Network changes dynamically. Road closures and openings happen at random due to conflicting decisions of authorities. How can we efficiently manage such updates in our communication infrastructure?

3. **Research Questions:**

- a. Can edge weights be updated by an increment $\Delta > 1$ in a single operation across a range?
- b. Can edge insertions and deletions be performed in $O(\log n)$ without rebuilding the structure?
- c. Can lexicographically ordered (u, v) edge pairs be queried and scanned efficiently?
- d. Can both adjacency list and adjacency matrix representations be supported, alongside $N(k)$ queries?
- e. Can full dynamic graph maintenance (connectivity + range) be sustained simultaneously?

4. **Literature Review:**

Paper	Year	Tools	Resolution	Gap
D-Tree (Chen, Lachish, Helmer, Böhlen)	2022	Spanning trees, union-find, adjacency lists	Fast average-case dynamic connectivity on large real-world graphs	No range queries; no bulk updates; no lexicographic edge ordering; no k-hop support

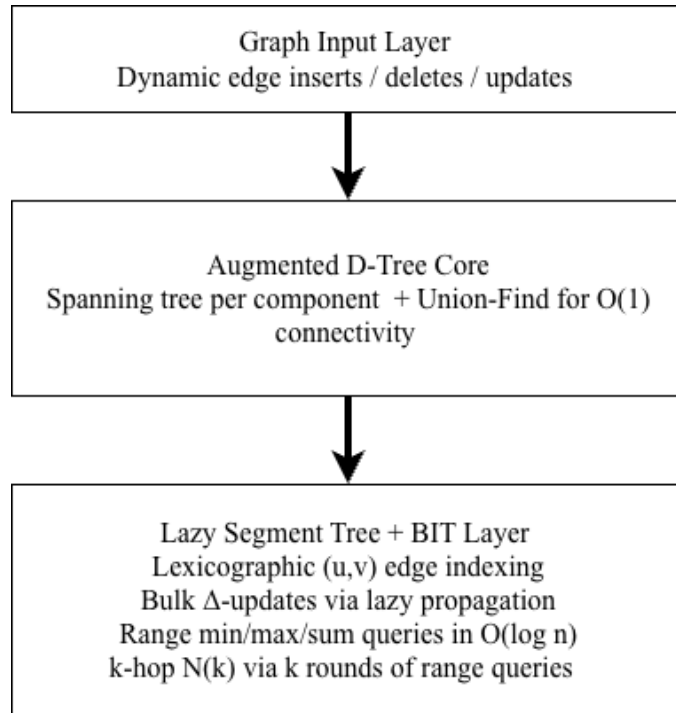
DS-Tree (Xu, Zhang, Wen, Qin, Zhang, Lin)	2024	Dual spanning tree + disjoint-set (union-find)	O(1) connectivity queries; best-known query time for dynamic undirected graphs	Higher update overhead; still no range query layer; single-edge operations only
Dynamic Segment Tree for Ranges and Prefixes	2003	Balanced BST augmented with range sets, endpoint scheme	Dynamic range insert/delete/query without needing all keys in advance	Not applied to graphs; no lazy propagation; no edge-pair key ordering
BITS-Tree (Easwarakumar, Hema)	2015	Modified segment tree with extended interval nodes	Faster dynamic segment insertion/deletion; faster stabbing and range queries than standard segment trees	Not graph-aware; no lazy bulk update; no integration with connectivity structures
Concurrent Dynamic Connectivity (Fedorov, Koval, Alistarh)	2021	Euler Tour Trees, concurrent spanning forest	First truly concurrent dynamic connectivity with sequential-equivalent time complexity	Complex implementation; no range query primitives; not benchmarked on standard datasets

What all existing work shares (and lacks)

Feature	D-Tree(2022)	DS-Tree(2024)	Proposed Approach
Dynamic edge insert/delete	Yes	Yes	Yes
Connectivity query	Yes	Yes, O(1)	Yes
Range query on edges	No	No	Yes, O(log n)
Bulk update ($\Delta > 1$)	No	No	Yes, Lazy Loop
Lexicographic edge order	No	No	Yes

k-hop neighbourhood queries	No	No	Yes
Adj. matrix + list support	List Only	List Only	Both

5. **Proposed Approach:**



Complexity Summary

Operation	Before (D-Tree/DS-Tree)	After (Range Graph)
Connectivity query	$O(1)$	$O(1)$
Single edge insert/delete	$O(\log^2 n)$ amortized	$O(\log n)$
Point edge weight update	$O(\log n)$	$O(\log n)$
Bulk range update (Δ on range)	Not supported	$O(\log n)$
Range min/max/sum query	Not supported	$O(\log n)$
Lexicographic edge scan	Not supported	$O(k \log n)$
k-hop $N(k)$ query	$O(k(n+m))$	$O(k \log n)$

6. Datasets:

Datasets	Source	Scale	Type	Purpose	Availability
SNAP Road Networks	Stanford SNAP	~1M nodes, 2.7M edges	Real-world	Simulate dynamic edge changes via timestamped road events; primary benchmark	snap.stanford.edu .
DIMACS Challenge Graphs	DIMACS	10K – 10M nodes	Standard benchmark	Direct comparison against D-Tree and DS-Tree published results	dimacs.rutgers.edu .
DBLP Co-authorship	Stanford SNAP	~317K nodes, 1M edges	Real-world	Dynamic edge insertions as new papers are published; tests sparse update patterns	snap.stanford.edu .
LiveJournal Social Graph	Stanford SNAP	~4M nodes, 35M edges	Stress test	Tests scalability and bulk update performance at social-network scale	snap.stanford.edu .
Synthetic (Erdős–Rényi)	Generated	Variable	Controlled	Isolates specific parameters: graph density, update rate, range width	

7. Workflow:

Weeks	Phases	Tasks	Deliverables
1-2	Baseline Reproduction	Read D-Tree & DS-Tree papers (use summaries above) · Clone D-Tree GitHub code · Run on SNAP Road Network dataset · Document observed performance and structural weaknesses	Baseline system running + written weakness analysis
3-4	Range Query Layer	Implement lazy segment tree from scratch · Unit-test on standalone arrays · Design (u,v) lexicographic key scheme for edges · Verify $O(\log n)$ bulk update correctness	Standalone segment tree with lazy propagation — tested and documented
5-6	Integration	Augment D-Tree with segment tree layer · Synchronize both structures on every insert/delete · Implement range edge queries · Implement lexicographic edge scanning	Integrated RangeGraph prototype — D-Tree core + range layer working together
7-8	Full Feature Set	Add lazy bulk Δ -updates (RQ1) · Implement k-hop $N(k)$ via range query rounds (RQ4) · Add 2D BIT for adjacency matrix support · All 5 research questions answered in working code	All RQs implemented and tested · Performance measurements begin

9-10	Benchmarking + Paper	Run experiments comparing RangeGraph vs D-Tree vs DS-Tree on SNAP + DIMACS · Plot and analyse results · Write paper in IEEE two-column format · Finalize slides for submission	Submitted paper + benchmark results + presentation slides
------	----------------------	--	---